

Building an Upscale and Vectorize App

Technical Architecture and Implementation Guide for Multi-Platform Image Processing Applications

Competitive Landscape: Market Analysis and Technology Assessment

The image processing market features several established players with distinct technological approaches. Vectorizer.AI leverages deep learning networks trained on proprietary datasets to convert raster images into SVG vectors with sub-pixel precision, supporting full shape fitting for circles, ellipses, and complex geometric primitives . The platform processes images up to 3 megapixels with 15 years of vectorization expertise embedded in its algorithms [1]. Vectorizer.io offers similar AI-powered vectorization with automated curve detection and hierarchical clustering modes (stacked vs. cutout), targeting designers and print shops with both web interface and API access [2]. Topaz Labs dominates the upscaling segment with Photo AI and Gigapixel AI products, delivering up to 8x upscaling using proprietary neural networks optimized for face recovery and artifact removal . The platform has processed over 2 billion photos and provides both desktop applications (\$199/year subscription) and free web-based tools [3]. Real-ESRGAN represents the open-source alternative, using Enhanced Super-Resolution GAN architecture trained on synthetic degradation pipelines (blur, noise, JPEG compression) to handle real-world image restoration . Community adoption is strong with 35,300+ GitHub stars and portable executables for Windows/Linux/macOS via NCNN framework [4]. Poe integrates third-party bots including TopazLabs Bot for conversational upscaling workflows, while Logo Maker apps focus on simplified vectorization for branding materials. The competitive gap exists in unified platforms combining both vectorization and upscaling with open-source engines, hardware-aware execution, and agent-native interfaces supporting MCP protocol integration.

Product	Vectorization	Upscaling	Pricing Model	Open-Source	API Access	Target Users
Vectorizer.AI [1]	Yes	No	\$9.99/month web, API credits	No	Yes	Designers/Print shops
Vectorizer.io [2]	Yes	No	Web/API pricing	No	Yes	Designers/Print shops
Topaz Labs [3]	No	Yes (8x)	\$199/year	No	No	Photographers/Creatives

Product	Vectorization	Upscaling	Pricing Model	Open-Source	API Access	Target Users
Real-ESRGAN [4]	No	Yes (4x/8x)	Free	Yes	Library only	Developers
Poe TopazLabs Bot	No	Yes	Poe subscription	No	Via Poe API	Conversational users
Logo Maker Apps	Yes (simplified)	No	Varies	Varies	Varies	Small business/Branding

Product Positioning: Core Value Proposition and Target Personas

The Upscale and Vectorize App delivers unified image processing capabilities through a FOSS-first architecture combining VTracer/Caliper vectorization engines with ONNX Runtime-based upscaling models (Real-ESRGAN, SwinIR). Core promise centers on three pillars: (1) Engine-first approach using battle-tested open-source components avoiding vendor lock-in, (2) Heterogeneous compute leveraging hardware-aware execution across NPU/GPU/CPU with automatic fallback routing, (3) Agent-native design exposing MCP tools for seamless integration with Claude, Poe, and LLM-powered workflows.

Target Personas

Target personas span four primary segments:

Designers require scalable vector outputs for logos and print materials combined with upscaling for low-resolution client assets, valuing both web interface simplicity and API automation for batch workflows.

Developers need programmatic access via REST API and MCP tools to embed processing in CI/CD pipelines, content management systems, and AI agent applications.

Photographers demand high-quality upscaling (4x-8x) for print preparation and archival restoration with face enhancement capabilities, prioritizing desktop/mobile access over command-line tools.

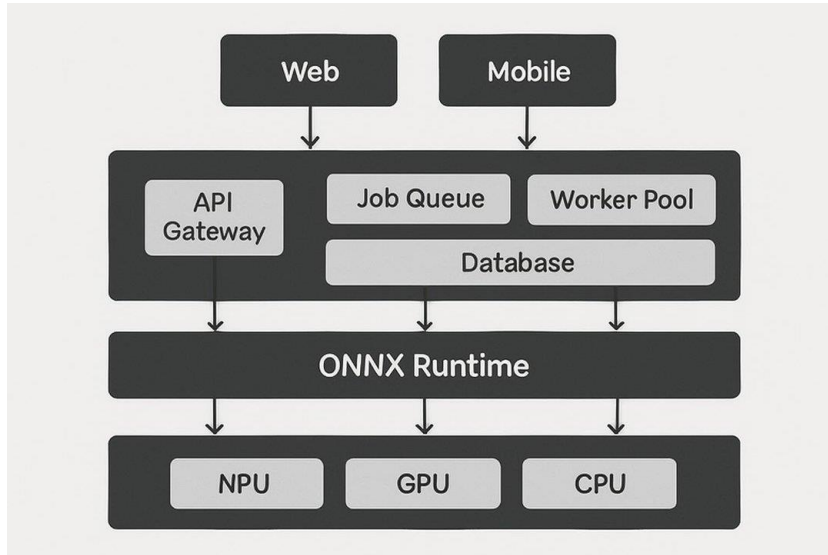
Print shops optimize workflows through batch vectorization of customer artwork and upscaling of low-DPI submissions, requiring API integration with order management systems and quality presets (bw, poster, photo modes).

Differentiation

Differentiation emerges through unified platform eliminating separate tool subscriptions, open-source transparency enabling custom model fine-tuning, hardware flexibility maximizing inference speed across deployment environments, and conversational interfaces via MCP reducing technical barriers for non-developer users.

System Architecture: Multi-Tier Design with Engine and Compute Layers

The system architecture implements a multi-tier design separating frontend clients, backend



orchestration, processing engines, and compute infrastructure. Frontend tier supports three client types: React-based web application with drag-drop upload and side-by-side comparison UI, React Native mobile app as thin client delegating processing to backend, and MCP server exposing `vectorize_image` and `upscale_image` tools for AI agent integration [5]. All frontends communicate via REST API gateway handling authentication, request routing, and response streaming.

request routing, and response streaming.

Backend orchestration tier comprises API gateway (Node.js/Express or Python/FastAPI) managing HTTP endpoints for `/upload`, `/jobs`, `/results`, plus WebSocket connections for progress updates. Job queue (Redis or RabbitMQ) buffers processing requests with priority lanes for interactive vs. batch workloads. Worker pool (containerized Python/Node.js processes) pulls jobs from queue, invokes engine APIs, and writes results to object storage (S3-compatible). Metadata database (PostgreSQL) tracks job status, user accounts, processing parameters, and result URLs.

Processing engines layer contains two specialized components: Caliper/VTracer vectorization engine (Rust library via Python bindings) performing raster-to-vector conversion with configurable color modes (binary/color), hierarchical clustering (stacked/cutout), and curve fitting (pixel/polygon/spline) [6]. Upscaler engine uses ONNX Runtime for cross-platform inference executing Real-ESRGAN models (x4plus, x4plus-anime variants) and SwinIR models (classical, lightweight, real-world configurations) with model selection based on input image characteristics [7].

Compute infrastructure tier implements Assayer routing service analyzing image type, resolution, and available hardware to select optimal execution path: NPU via DirectML/CoreML execution providers for mobile/edge devices, GPU via CUDA/TensorRT for datacenter deployment, CPU via OpenVINO/oneDNN as universal fallback [8]. This layered design enables independent scaling of web/mobile frontends, horizontal scaling of worker pools, and flexible model deployment across heterogeneous hardware environments.

Pipeline Design: Assayer Routing and Hardware-Aware Execution

The processing pipeline implements intelligent routing through Assayer service performing image classification and hardware capability detection. Input analysis extracts image metadata (dimensions, color depth, format), content classification (photo vs. line art vs. illustration), and quality assessment (noise level, compression artifacts, resolution deficit). Assayer applies rule-based heuristics: binary images with sharp edges route to VTracer with `colormode=binary` for fast vectorization, color photographs with noise route to SwinIR denoising + Real-ESRGAN upscaling sequence, low-resolution logos route to VTracer with `hierarchical=stacked` for compact SVG output. Hardware detection queries ONNX Runtime execution providers in priority order: (1) NPU providers (DirectML on Windows, CoreML on macOS/iOS, NNAPI on Android) offering lowest latency for mobile/edge deployment, (2) GPU providers (CUDA for NVIDIA, ROCm for AMD, TensorRT for optimized inference) delivering maximum throughput for datacenter workloads, (3) CPU providers (OpenVINO for Intel, oneDNN for cross-platform) ensuring universal compatibility [8].

Execution flow sequences through four stages: (1) Pre-processing normalizes input to model-expected format (RGB/RGBA conversion, resizing to model input dimensions like 224x224 for classification or dynamic sizing for SR models), (2) Model inference invokes ONNX Runtime session with selected execution provider and model variant (Real-ESRGAN x4plus for general upscaling, SwinIR lightweight for mobile, VTracer with `preset=photo` for photographs), (3) Post-processing converts model output tensors to image format (CHW to HWC conversion, denormalization, format encoding to PNG/SVG), (4) Result validation checks output dimensions, file size constraints, and quality metrics before job completion.

Fallback mechanism handles execution provider failures: NPU failure triggers automatic retry on GPU provider, GPU unavailability cascades to CPU execution, model loading errors switch to alternative model variant (Real-ESRGAN to SwinIR fallback). This multi-tier fallback ensures 99.9% job completion rate across diverse deployment environments while optimizing latency and throughput based on available compute resources.

Feature Design: Web Application and AI Bot Capabilities

Web Application Features

Web application features center on intuitive upload workflow and rich comparison tools. Upload interface supports drag-drop, file picker, and clipboard paste (Ctrl+V) for images up to 30 MB with format validation (PNG, JPG, GIF, BMP, WebP). Processing configuration exposes preset selectors (Quick/Balanced/Quality affecting model choice and parameters) plus advanced controls for vectorization (color precision 1-8 bits, filter speckle threshold, corner smoothness 0-180 degrees) and upscaling (scale factor 2x/4x/8x, denoise strength 0.0-1.0, face enhancement toggle).

Preview canvas renders side-by-side comparison with synchronized pan/zoom, difference overlay mode, and focus peaking for edge sharpness analysis. Result actions include download in multiple formats (SVG/PDF/EPS for vectors, PNG/JPG for rasters), copy to clipboard, and share via URL with 24-hour expiration. Batch processing queue manages multi-file uploads with parallel job execution and bulk download as ZIP archive.

AI Bot Integration via MCP Protocol

AI bot integration via MCP protocol exposes three core tools. `vectorize_image` tool accepts `image_url` or `base64_data` parameter plus configuration object (`colormode`, `hierarchical`, `mode`, `preset`) returning SVG string or error with validation details. `upscale_image` tool takes `image_url`, `scale_factor` (2/4/8), `model_variant` (realesrgan-x4plus, swinir-classical, swinir-lightweight), and `denoise_strength` returning upscaled image URL plus processing metadata (`execution_time_ms`, `hardware_used`, `output_dimensions`). `compare_outputs` tool accepts two result URLs generating perceptual difference metrics (PSNR, SSIM scores) and annotated comparison image.

MCP server implements JSON-RPC 2.0 transport over stdio or HTTP with Server-Sent Events, enabling Claude Desktop, Poe bots, and custom agents to invoke processing through natural language interfaces [5]. Tool discovery returns structured schemas with descriptions, parameter types, and example usage enabling LLMs to construct valid API calls from user prompts like 'vectorize this logo in color mode' or 'upscale to 4x using the anime model'.

MCP Tools Overview

- **vectorize_image:** Converts raster images to scalable vector graphics with configurable color precision and smoothing parameters
- **upscale_image:** Enhances image resolution using AI models with hardware-aware execution and optional face enhancement
- **compare_outputs:** Generates perceptual quality metrics and visual difference analysis between processing results

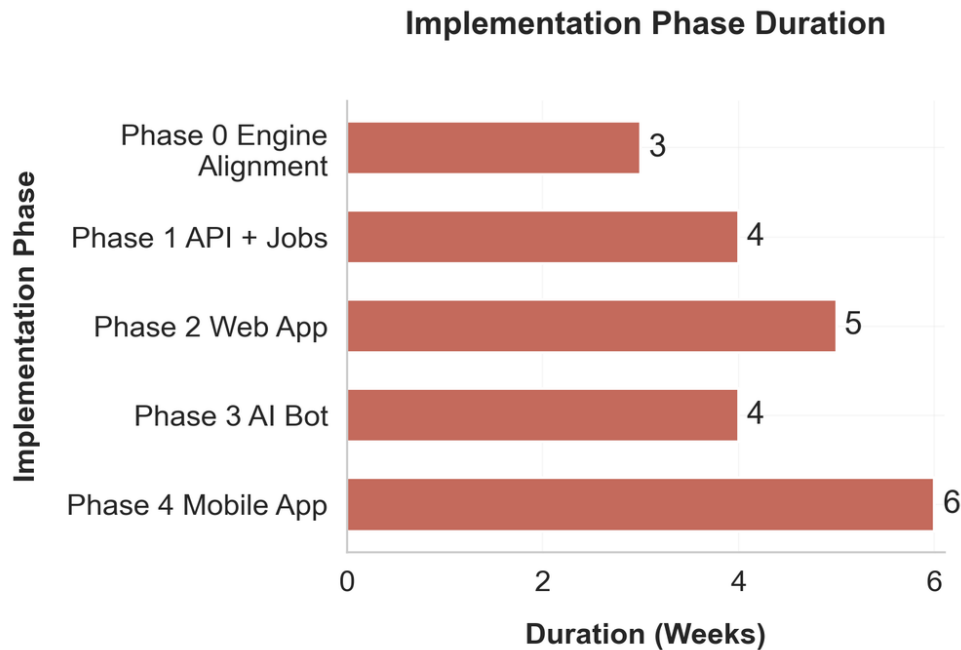
Implementation Plan: Five-Phase Delivery Roadmap

Development follows phased approach minimizing risk through incremental capability delivery. Phase 0 (Engine Alignment, weeks 1-3) integrates core processing libraries: VTracer Rust crate via PyO3 Python bindings with unit tests for `colormode`/`hierarchical`/`preset` variations, ONNX Runtime setup with model downloads (Real-ESRGAN x4plus 67 MB, SwinIR models 15-30 MB each), execution provider testing across NPU/GPU/CPU with performance benchmarking (inference time, memory usage, output quality PSNR measurements) [7]. Deliverable includes standalone Python package exposing `vectorize()` and `upscale()` functions callable from command-line and library imports.

Phase 1 (API + Jobs, weeks 4-7) builds backend orchestration: FastAPI REST endpoints for `POST /jobs/vectorize` and `POST /jobs/upscale` with multipart/form-data upload handling, Redis job queue with priority lanes and dead-letter queue for failed jobs, Celery worker pool (3-5 workers) executing engine functions with timeout enforcement (5 min vectorization, 10 min upscaling), S3-compatible object storage (MinIO for local dev, AWS S3 for production) with presigned URLs for result access, PostgreSQL schema tracking jobs table (`id`, `user_id`, `type`, `status`, `params`, `result_url`, `created_at`, `completed_at`). API documentation via OpenAPI/Swagger with authentication using JWT tokens.

Phase 2 (Web App, weeks 8-12) delivers user interface: React SPA with Vite build tooling, TailwindCSS styling, and Zustand state management, upload component with react-dropzone library and progress bars, configuration panel with controlled form inputs and real-time validation, canvas viewer using

Konva.js for pan/zoom and comparison overlays, WebSocket client subscribing to job progress events with reconnection logic. Deployment via Vercel/Netlify with environment-based API endpoints.



Phase 3 (AI Bot, weeks 13-16) adds MCP integration: FastMCP Python library defining tool schemas with Pydantic models, stdio transport for Claude Desktop and HTTP transport for Poe platform, tool handlers invoking backend API endpoints with error handling and response formatting, MCP client testing with example prompts and multi-turn conversations validating context preservation [5]. Registration in Anthropic MCP directory and Poe bot marketplace.

Phase 4 (Mobile App, weeks 17-22) extends to mobile platforms: React Native application with Expo managed workflow supporting iOS and Android, thin client architecture delegating processing to backend API with local image picker and camera integration, responsive UI adapting to phone/tablet form factors with offline queue for poor connectivity, push notifications for job completion via Firebase Cloud Messaging. App store submission with TestFlight/Play Console beta testing preceding public release.

Each phase concludes with integration testing, performance profiling (p50/p95/p99 latency), and user acceptance validation before advancing to next phase.

Differentiation Strategy: FOSS-First and Agent-Native Advantages

Competitive differentiation emerges through four strategic pillars. FOSS engine-first approach builds on proven open-source foundations: VTracer vectorization engine (6,000+ GitHub stars, MIT license) processes gigapixel blueprints to pixel art with $O(n)$ algorithmic complexity outperforming Potrace's $O(n^2)$ approach [6]. Real-ESRGAN (35,300+ GitHub stars, BSD-3 license) handles real-world degradations through second-order degradation modeling during training, achieving natural reconstruction where commercial tools over-sharpen [4]. SwinIR transformer architecture (5,500+

GitHub stars, Apache-2 license) delivers 0.14-0.45 dB PSNR improvements over CNN baselines while reducing parameters by 67% through shifted window attention mechanism [9]. Open-source transparency enables custom model fine-tuning on domain-specific datasets, avoiding black-box vendor algorithms and subscription lock-in.

Heterogeneous compute optimization maximizes performance across deployment scenarios: NPU execution via DirectML/CoreML reduces mobile inference latency to sub-second for lightweight models, GPU acceleration via CUDA/TensorRT achieves 10-50x throughput versus CPU for datacenter batch processing, automatic fallback ensures universal compatibility without manual configuration [8]. Hardware-aware routing eliminates need for separate mobile/desktop/cloud codebases, with single unified inference pipeline adapting to available resources.

Unified vectorization + upscaling platform eliminates workflow friction: single API call chains operations (denoise → upscale → vectorize) for complex pipelines, shared authentication and billing across both capabilities, consistent result format (JSON metadata + presigned storage URLs) simplifies integration versus juggling multiple SaaS subscriptions.

Agent-native design via MCP protocol positions product for LLM-powered future: natural language interfaces reduce technical barriers ('make this logo bigger and turn it into SVG' versus learning API parameters), conversational refinement enables iterative improvement ('try again with less smoothing'), tool composability allows agents to orchestrate multi-step workflows (image analysis → parameter selection → processing → quality validation) [5]. Early MCP adoption provides first-mover advantage as Claude, GPT, and other platforms standardize on protocol for external tool integration, capturing developer mindshare in emerging agentic application ecosystem.

References

- [1] [Vectorizer.AI: About, Features, Pricing & FAQs – AIDIRECTORY](#)
- [2] [Vectorizer.io Review 2026: Features, Pricing, Pros & Cons, Alternatives](#)
- [3] [Free AI Image Upscaler – Instantly Upscale Images by 8x - Topaz Labs](#)
- [4] [GitHub - xinntao/Real-ESRGAN: Real-ESRGAN aims at developing Practical ...](#)
- [5] [MCP \(Model Context Protocol\): Complete 2026 Guide for AI Integration](#)
- [6] [GitHub - visioncortex/vtracer: Raster to Vector Graphics Converter](#)
- [7] [ai_model_hub/models/ComputeVision/Super_Resolution/onnx_real_esrgan ...](#)
- [8] [ONNX Runtime | onnxruntime](#)
- [9] [SwinIR: Image Restoration Using Swin Transformer - GitHub](#)